

Manual: Middleware, Mensageria e Observabilidade

WildFly/JBoss EAP, Mensageria e EIP, ActiveMQ, Kafka, CDC e Debezium, Kubernetes avançado, Ansible, Helm, Prometheus, Grafana, Zabbix, ELK, APM e OpenTelemetry

CHANCELA EDITORIAL YLUNA85 LABS

Preparação Conjunta: Auditor Fiscal (TI) & Agente de Tributos Estaduais

Revisão Ortográfica e Glossário Geral de Siglas

Material de livre circulação interna. Revisado em Português Brasileiro (pt-BR).

1. Middleware: Conceito, Tipos e Acoplamento

Middleware é a camada de software entre o sistema operacional e as aplicações que esconde a heterogeneidade de redes, sistemas e linguagens e oferece serviços comuns de comunicação e integração para aplicações distribuídas.

- **Tipos clássicos de middleware**

Chamada de procedimento remoto (RPC); orientado a mensagens (MOM), como filas e tópicos; orientado a objetos (como CORBA e Java RMI); monitores de processamento de transações; servidores de aplicação; middleware de acesso a dados (como JDBC e ODBC); barramento de serviços (ESB) e, em arquiteturas atuais, API gateways.

- **Comunicação síncrona e assíncrona**

Na síncrona (por exemplo, requisição e resposta HTTP), o cliente espera o retorno. Na assíncrona (mensageria), o emissor envia e segue o trabalho, e o receptor processa depois. A assíncrona reduz o acoplamento e absorve picos de carga, ao custo de mais complexidade (ordem, duplicidade, consistência eventual).

- **Tipos de acoplamento**

Espacial (emissor e receptor não precisam se conhecer), temporal (não precisam estar ativos ao mesmo tempo) e de sincronização (o emissor não fica bloqueado). Mensageria com filas e tópicos oferece esses desacoplamentos.

- **Broker de mensagens**

Componente intermediário que recebe, armazena (opcionalmente em disco) e encaminha mensagens entre produtores e consumidores, cuidando de roteamento, persistência, confirmação e reentrega.

ASSÍNCRONO NÃO É SINÔNIMO DE RÁPIDO

Mensageria melhora a resiliência e a escalabilidade, mas não garante menor latência. O ganho está no desacoplamento.

2. Servidores de Aplicação Jakarta EE: WildFly e JBoss EAP

Servidores de aplicação hospedam aplicações Java corporativas e fornecem serviços como transações, persistência, mensageria e segurança. A plataforma Java EE passou a ser mantida pela Eclipse Foundation com o nome Jakarta EE.

- **Jakarta EE e o namespace**

Jakarta EE 8 manteve os pacotes `javax.*`. A partir do Jakarta EE 9, os pacotes passaram para `jakarta.*`, o que exige migração de aplicações ao mudar de versão do servidor. Especificações típicas: Servlet, Faces, CDI, Persistence (JPA), Messaging (JMS), Transactions (JTA), RESTful Web Services (JAX-RS) e Enterprise Beans (EJB).

- **WildFly e JBoss EAP**

WildFly é o servidor de aplicação de código aberto (o antigo JBoss AS, renomeado em 2014). O Red Hat JBoss Enterprise Application Platform (EAP) é o produto com suporte comercial, construído a partir do WildFly.

- **Modos de operação**

Standalone: uma instância, configurada em `standalone.xml` (há variantes como `standalone-ha.xml` para cluster e `standalone-full.xml` para mensageria). Managed Domain: um controlador de domínio gerencia vários servidores por meio de `domain.xml` e `host.xml`, com implantação e configuração centralizadas.

- **Administração**

Interface de linha de comando (`jboss-cli`), console web e API de gerenciamento, na porta 9990. A aplicação responde em HTTP, por padrão, na porta 8080. Aplicações (WAR, EAR ou JAR) são implantadas pela CLI, pelo console ou pelo diretório de implantação no modo standalone. O deslocamento de portas (port offset) permite várias instâncias no mesmo host.

- **Subsistemas e arquitetura interna**

A configuração é organizada em subsistemas: web (Undertow), datasources, cache e clustering (Infinispan e JGroups), mensageria (ActiveMQ Artemis), transações (Narayana), logging e segurança (Elytron). O carregamento de classes é modular (JBoss Modules), o que isola dependências entre aplicações.

- **Tomcat versus servidor completo**

O Apache Tomcat é um contêiner de servlets e páginas web (implementa parte das especificações, como Servlet e JSP), sem a plataforma Jakarta EE completa. WildFly e JBoss EAP implementam a plataforma completa (ou perfis dela).

STANDALONE É UMA INSTÂNCIA, DOMAIN É GESTÃO CENTRALIZADA

No modo domain, um domain controller aplica a configuração a vários servidores e grupos de servidores. No standalone, cada instância é independente. Console de gestão: porta 9990; aplicação: 8080.

3. Fundamentos de Mensageria: Filas, Tópicos e Garantias de Entrega

A mensageria assíncrona usa um broker para trocar mensagens entre produtores e consumidores. A API Jakarta Messaging (JMS) padroniza o acesso a brokers em Java, e há protocolos abertos como AMQP, MQTT e STOMP.

- **Fila (ponto a ponto) e tópico (publicação e assinatura)**
Em uma fila, cada mensagem é entregue a um único consumidor (com vários consumidores concorrentes, cada um recebe parte das mensagens). Em um tópico, cada assinante recebe uma cópia da mensagem. Assinaturas duráveis retêm mensagens enquanto o assinante está desconectado.
- **Anatomia da mensagem**
Cabeçalhos (como identificador, destino, modo de entrega, prazo de validade e prioridade), propriedades (metadados definidos pela aplicação, usados também em seletores) e corpo (payload). Modo de entrega persistente grava em disco; não persistente prioriza velocidade.
- **Confirmação e reentrega (JMS)**
Modos de confirmação: AUTO_ACKNOWLEDGE, CLIENT_ACKNOWLEDGE e DUPS_OK_ACKNOWLEDGE; a sessão transacional (SESSION_TRANSACTED) agrupa envio e recebimento em uma transação. Se a confirmação não ocorre, o broker reentrega a mensagem. Mensagens que falham repetidamente vão para uma fila de mensagens mortas (DLQ, dead letter queue).
- **Semânticas de entrega**
No máximo uma vez (at-most-once): pode perder, não duplica. Pelo menos uma vez (at-least-once): não perde, pode duplicar. Exatamente uma vez (exactly-once): sem perda e sem duplicidade, difícil de garantir de ponta a ponta; na prática usa-se entrega pelo menos uma vez com consumidores idempotentes ou mecanismos transacionais.
- **Ordem e concorrência**
Consumidores concorrentes aumentam a vazão, mas podem quebrar a ordem global das mensagens. Garantia de ordem costuma ser por fila, por partição ou por grupo de mensagens, conforme o broker.
- **Transações distribuídas**
JMS pode participar de transações XA (dois fases, two-phase commit) junto com bancos de dados, com custo de desempenho e complexidade. Alternativas modernas usam padrões como outbox e sagas.

DUPLICIDADE É NORMAL, IDEMPOTÊNCIA É A DEFESA

Com entrega pelo menos uma vez, o mesmo evento pode chegar duas vezes. O consumidor idempotente produz o mesmo resultado ao processar a mensagem repetida.

4. Padrões de Integração Corporativa (Enterprise Integration Patterns)

O catálogo Enterprise Integration Patterns (EIP), de Hohpe e Woolf, descreve soluções recorrentes para integração por mensagens. Frameworks como o Apache Camel implementam esses padrões. O programa de TI cita os grupos de mensageria, roteamento e transformação.

- **Canais de mensagem**

Message Channel (canal), Point-to-Point Channel, Publish-Subscribe Channel, Dead Letter Channel (mensagens não entregáveis) e Guaranteed Delivery (persistência para garantir a entrega).

- **Roteamento**

Content-Based Router (decide o destino pelo conteúdo), Message Filter (descarta mensagens que não interessam), Recipient List (envia a vários destinatários), Splitter (divide uma mensagem em partes), Aggregator (junta mensagens relacionadas) e Resequencer (reordena).

- **Transformação**

Message Translator (converte formatos), Content Enricher (acrescenta dados de outra fonte), Content Filter (remove campos), Envelope Wrapper, Claim Check (guarda o conteúdo grande fora e passa só uma referência), Normalizer e Canonical Data Model (modelo de dados comum).

- **Endpoints de mensageria**

Competing Consumers (vários consumidores disputam mensagens da mesma fila), Polling Consumer e Event-Driven Consumer, Durable Subscriber, Idempotent Receiver e Messaging Gateway.

- **Correlação e resposta**

Correlation Identifier (liga resposta à requisição), Return Address (onde enviar a resposta) e Request-Reply (requisição e resposta sobre canais assíncronos).

ROUTER DECIDE O DESTINO, TRANSLATOR MUDA O FORMATO

Roteamento (router, filter, splitter, aggregator) trata de para onde a mensagem vai. Transformação (translator, enricher) trata de como a mensagem é representada.

5. Apache ActiveMQ: Classic e Artemis

O Apache ActiveMQ é um broker de mensagens de código aberto com suporte a JMS e a vários protocolos. Existem dois produtos: ActiveMQ Classic (série 5.x) e ActiveMQ Artemis, de nova geração.

- **Classic e Artemis**
O Artemis, de nova geração, tem origem no código do HornetQ, doado ao projeto Apache, e é o broker de mensageria embutido nos servidores WildFly e JBoss EAP. O Classic é o broker tradicional, com longa base instalada.
- **Protocolos suportados**
JMS, AMQP, MQTT, STOMP e OpenWire (protocolo nativo do ActiveMQ Classic, também suportado pelo Artemis).
- **Portas padrão**
Porta 61616 para conexões de mensagens (OpenWire no Classic; aceitador multiprotocolo no Artemis) e 8161 para o console web.
- **Persistência**
Classic: KahaDB é o armazenamento padrão. Artemis: journal de gravação sequencial (append-only), com opção de armazenamento em banco de dados por JDBC.
- **Modelo de endereçamento no Artemis**
Um endereço (address) tem filas associadas e um tipo de roteamento: anycast (uma fila recebe cada mensagem, modelo ponto a ponto) e multicast (cada assinante recebe uma cópia, modelo de tópico).
- **Alta disponibilidade e escalabilidade**
Pares ativo/backup com armazenamento compartilhado ou replicação; agrupamento de brokers em rede (network of brokers no Classic, cluster no Artemis) para distribuir carga. No Classic há tópicos virtuais (virtual topics), que combinam a semântica de tópico com filas por consumidor.

ARTEMIS É O BROKER DO WILDFLY

Na configuração do WildFly e do JBoss EAP, o subsistema de mensageria é o ActiveMQ Artemis. Não confundir com o ActiveMQ Classic (5.x).

6. Apache Kafka: Arquitetura

O Apache Kafka é uma plataforma distribuída de streaming de eventos. Em vez de fila que remove a mensagem ao consumir, o Kafka mantém um registro (log) particionado, replicado e ordenado, do qual os consumidores leem na sua posição.

- **Conceitos centrais**

Evento (registro com chave, valor e data), tópico (categoria de eventos), partição (subdivisão ordenada de um tópico), offset (posição sequencial do registro na partição), broker (servidor Kafka) e cluster (conjunto de brokers). Produtores gravam; consumidores leem.

- **Partições e ordem**

A ordem é garantida apenas dentro de uma partição, não entre partições de um tópico. Registros com a mesma chave vão para a mesma partição (por padrão, pelo hash da chave), preservando a ordem por chave. Mais partições permitem mais paralelismo.

- **Replicação**

Cada partição tem um líder e réplicas seguidoras em outros brokers. Produtores e consumidores normalmente falam com o líder. As réplicas em dia formam o ISR (in-sync replicas). O fator de replicação define quantas cópias existem.

- **Grupos de consumidores**

Consumidores com o mesmo group.id dividem as partições do tópico: cada partição é lida por, no máximo, um consumidor do grupo. O paralelismo máximo do grupo é o número de partições. Grupos diferentes leem os mesmos dados de forma independente. Ao entrar ou sair consumidor, ocorre o rebalanceamento.

- **Offsets**

Cada grupo registra até onde leu (offset) no tópico interno `__consumer_offsets`. O consumidor pode reler de um offset anterior (reprocessamento). A política `auto.offset.reset` (earliest ou latest) define onde começar sem offset salvo.

- **Retenção**

O Kafka retém os registros por tempo ou tamanho (o padrão é de 7 dias), independentemente de terem sido lidos. A compactação de log (log compaction) mantém apenas o último valor de cada chave.

- **Metadados: ZooKeeper e KRaft**

Versões antigas dependiam do Apache ZooKeeper para metadados e eleição de controlador. O modo KRaft guarda os metadados no próprio Kafka (protocolo Raft); nas versões mais recentes o ZooKeeper deixou de ser suportado. Conferir a versão em uso. Porta padrão dos brokers: 9092.

ORDEM SÓ DENTRO DA PARTIÇÃO

Para manter a ordem de eventos de uma mesma entidade (por exemplo, um contribuinte), use a mesma chave, que leva os eventos à mesma partição. Não existe ordem global entre partições.

7. Apache Kafka: Garantias, Ecossistema e Comparação com Filas

As garantias de entrega dependem da configuração de produtores, brokers e consumidores.

- **Confirmação do produtor (acks)**

acks=0: não espera confirmação (pode perder). acks=1: espera o líder gravar. acks=all: espera as réplicas do ISR, e junto com min.insync.replicas oferece maior durabilidade.

- **Semânticas de entrega**

No consumidor, confirmar o offset antes de processar leva a no máximo uma vez; confirmar depois de processar leva a pelo menos uma vez. Exatamente uma vez no Kafka usa produtor idempotente e transações, com consumidores em modo read_committed.

- **Kafka Connect**

Framework para integrar o Kafka a sistemas externos, sem código, com conectores de origem (source, entrada de dados no Kafka) e de destino (sink, saída de dados do Kafka), executado em workers, inclusive distribuídos.

- **Kafka Streams e Schema Registry**

Kafka Streams é uma biblioteca Java para processar fluxos (filtros, agregações e junções com estado). O Schema Registry guarda e valida esquemas de mensagens (por exemplo, Avro, Protobuf e JSON Schema) para evolução compatível de formatos.

- **Kafka versus fila tradicional**

O Kafka mantém o histórico e permite releitura (replay); o consumidor controla o offset (modelo pull); o desempenho vem da escrita sequencial em disco e do uso do cache de páginas; escala por partições. Filas tradicionais (como as do ActiveMQ) removem a mensagem após a confirmação e oferecem recursos como seletores, prioridades e roteamento mais rico.

- **Replicação entre clusters**

O MirrorMaker 2 replica tópicos entre clusters, por exemplo para recuperação de desastres.

KAFKA GUARDA, FILA ENTREGA E REMOVE

No Kafka, ler não apaga o registro: ele permanece até a retenção expirar e pode ser relido. Em uma fila tradicional, a mensagem confirmada some.

8. Captura de Dados de Alteração (CDC) e Debezium

CDC (Change Data Capture) captura as alterações feitas em um banco de dados (inserções, atualizações e exclusões) e as entrega a outros sistemas quase em tempo real. Debezium é uma plataforma de código aberto para CDC baseada em log.

- **Abordagens de CDC**

Baseada em log de transações (lê o log do banco, sem sobrecarga nas consultas e captura exclusões); baseada em gatilhos (triggers, adiciona carga nas transações); baseada em consulta ou em carimbo de data (polling, não captura exclusões e pode perder estados intermediários).

- **Debezium e Kafka Connect**

Os conectores do Debezium normalmente rodam no Kafka Connect como conectores de origem e publicam cada mudança em tópicos do Kafka. Há também o Debezium Server e o modo embutido. Bancos suportados incluem MySQL, PostgreSQL, SQL Server, Oracle, MongoDB e Db2.

- **Log lido em cada banco**

MySQL: log binário (binlog, com formato ROW). PostgreSQL: WAL por decodificação lógica, com slot de replicação e wal_level=logical. SQL Server: tabelas de CDC do próprio banco. Oracle: LogMiner ou XStream.

- **Estrutura do evento**

Cada evento traz o estado anterior (before), o estado posterior (after), metadados da origem (source) e a operação (op): c (criação), u (atualização), d (exclusão) e r (leitura, gerada no snapshot inicial).

- **Snapshot inicial**

Na primeira execução, o conector pode fazer um snapshot da situação atual das tabelas e depois passar a ler o log de alterações, para que o destino comece consistente.

- **Garantias e usos**

O Debezium oferece entrega pelo menos uma vez, então consumidores devem tolerar duplicidade. Usos: replicação para data warehouse e data lake, invalidação de cache, auditoria, integração entre microsserviços e padrão outbox (eventos gravados em tabela na mesma transação e publicados por CDC).

CDC POR LOG VÊ EXCLUSÕES E NÃO SOBRECARREGA

CDC baseado em log captura inclusive exclusões e evita consultas periódicas ao banco. Consultas por data (polling) não enxergam registros excluídos.

9. Kubernetes: Recursos Avançados de Operação

Complementa a seção de Kubernetes da apostila de Infraestrutura, com objetos e práticas usados na operação de aplicações.

- **Cargas de trabalho**
Deployment (aplicações sem estado, com atualização gradual e reversão), StatefulSet (identidade de rede estável e armazenamento persistente por réplica, implantação ordenada), DaemonSet (um pod por nó, como agentes de log e monitoramento) e Job e CronJob (execução única e agendada).
- **Sondas (probes)**
Liveness: se falhar, o contêiner é reiniciado. Readiness: se falhar, o pod deixa de receber tráfego do Service. Startup: protege aplicações de inicialização lenta.
- **Recursos e QoS**
Requests são a reserva usada pelo escalonador; limits são o teto imposto pelo runtime. Ao exceder o limite de CPU, o contêiner sofre throttling; ao exceder o de memória, pode ser encerrado (OOMKilled). Classes de QoS: Guaranteed, Burstable e BestEffort.
- **Escalonamento automático**
HPA (Horizontal Pod Autoscaler) ajusta o número de réplicas por métricas, como CPU, e depende do metrics-server. VPA ajusta requests e limits. O Cluster Autoscaler adiciona ou remove nós.
- **Configuração e segredos**
ConfigMap guarda configuração; Secret guarda dados sensíveis, mas seus valores são apenas codificados em base64, sem cifra por padrão. É recomendável habilitar cifra em repouso e usar gerenciadores de segredos e RBAC.
- **Armazenamento**
PersistentVolume (PV) é o volume; PersistentVolumeClaim (PVC) é o pedido de armazenamento pela aplicação; StorageClass define o tipo e permite provisionamento dinâmico. Modos de acesso: ReadWriteOnce, ReadOnlyMany e ReadWriteMany.
- **Segurança e isolamento**
Namespaces separam recursos; RBAC (Role, ClusterRole e bindings) controla permissões; NetworkPolicy restringe o tráfego entre pods (por padrão tudo é permitido, e a política exige um plugin de rede que a suporte).
- **Atualizações e operação**
Estratégia RollingUpdate com maxSurge e maxUnavailable; reversão com kubectl rollout undo; PodDisruptionBudget limita interrupções voluntárias; taints e tolerations, e afinidades, controlam onde os pods rodam. Desde a versão 1.24 o Docker Engine não é mais suportado diretamente como runtime; usam-se runtimes compatíveis com CRI, como containerd e CRI-O.

LIVENESS REINICIA, READINESS RETIRA DO TRÁFEGO

A sonda de liveness falha, o kubelet reinicia o contêiner. A de readiness falha, o pod sai dos endpoints do Service, sem reinício. Confundir as duas é erro comum.

10. Ansible, Helm, Podman e Padrões OCI

Automação e empacotamento complementam a orquestração: o Ansible automatiza a configuração de servidores, o Helm empacota aplicações para o Kubernetes, e o Podman e os padrões OCI atuam no nível dos contêineres.

- **Ansible: arquitetura**
O nó de controle executa o Ansible; os nós gerenciados normalmente exigem apenas acesso SSH e Python (no Linux) ou WinRM (no Windows), sem agente. O inventário lista os hosts, estático ou dinâmico.
- **Ansible: elementos**
Playbook (YAML) com plays e tasks; módulos; handlers (executados quando notificados, uma vez ao final); roles (estrutura reutilizável); variáveis e facts; templates Jinja2; Ansible Vault (cifra segredos); coleções. Os módulos command e shell não são idempotentes por si só.
- **Ansible: execução segura**
A opção --check simula sem alterar e --diff mostra as diferenças. O AWX e o Ansible Automation Platform oferecem interface, agendamento e controle de acesso.
- **Helm**
Gerenciador de pacotes do Kubernetes. Um chart reúne modelos (templates) e valores (values.yaml); a instalação de um chart gera uma release, que pode ser atualizada ou revertida (helm upgrade, helm rollback). O Helm 3 eliminou o componente Tiller.
- **Podman e Buildah**
Podman executa contêineres sem daemon central e com suporte a modo rootless, com CLI compatível com a do Docker e o conceito de pods. Buildah constrói imagens.
- **OCI (Open Container Initiative)**
Padrões abertos de imagem (Image Spec), de execução (Runtime Spec) e de distribuição (Distribution Spec). O runc é o runtime de referência. Docker, Podman, containerd e CRI-O usam esses padrões.

ANSIBLE DESCREVE ESTADO, HELM EMPACOTA

Ansible: configuração idempotente de servidores por SSH, sem agente. Helm: empacota e versiona aplicações do Kubernetes em charts. São ferramentas diferentes que se complementam.

11. Observabilidade: Logs, Métricas, Traces e Confiabilidade

Monitoramento verifica se o sistema está bem por meio de sinais conhecidos. Observabilidade é a capacidade de entender o estado interno do sistema a partir das saídas que ele produz, o que ajuda a investigar problemas inesperados.

- **Três pilares**

Logs (registros de eventos), métricas (medições numéricas ao longo do tempo) e traces (o caminho de uma requisição por vários serviços). Usados em conjunto, respondem o que aconteceu, quanto e onde.

- **Sinais dourados (SRE do Google)**

Latência, tráfego, erros e saturação.

- **Métodos RED e USE**

RED, para serviços: taxa de requisições (rate), erros e duração. USE, para recursos: utilização, saturação e erros.

- **SLI, SLO e SLA**

SLI é o indicador medido (por exemplo, percentual de requisições atendidas com sucesso). SLO é a meta interna para o SLI (por exemplo, 99,9%). SLA é o acordo formal com o cliente, com consequências. Orçamento de erros é o que sobra abaixo do SLO: para 99,9%, são 0,1% de falhas toleradas no período.

- **Boas práticas de logs**

Logs estruturados (por exemplo, JSON), níveis de severidade (DEBUG, INFO, WARN, ERROR), identificador de correlação ou de trace em cada registro e cuidado para não gravar dados pessoais ou segredos.

- **Alertas**

Alertar sobre sintomas que afetam o usuário (latência, erros) é mais útil que alertar sobre cada causa possível. Alertas devem ser acionáveis; excesso gera fadiga. Alta cardinalidade de rótulos em métricas aumenta muito o custo.

MÉTRICAS DIZEM QUANTO, TRACES DIZEM ONDE

Métrica mostra que a latência subiu. O trace mostra em qual serviço da cadeia o tempo foi gasto. O log explica o erro específico. Os três se complementam.

12. Prometheus, Grafana e Zabbix

Ferramentas de código aberto muito usadas para coleta de métricas, visualização e alertas.

- **Prometheus: modelo pull**

O Prometheus busca (scrape) as métricas por HTTP, em intervalos, no endpoint /metrics dos alvos. Exporters expõem métricas de sistemas que não as oferecem (o node_exporter expõe as do servidor Linux, na porta 9100). O servidor usa a porta 9090. Para jobs de curta duração há o Pushgateway.

- **Prometheus: dados e consultas**

Banco de séries temporais local, com métricas identificadas por nome e rótulos (labels). Linguagem PromQL (por exemplo, rate(), sum by() e histogram_quantile()). Retenção local padrão de 15 dias; armazenamento de longo prazo por remote write (Thanos, Mimir e similares).

- **Prometheus: tipos de métrica**

Counter (só cresce, como total de requisições), Gauge (sobe e desce, como memória em uso), Histogram (distribuição em faixas, permite percentis) e Summary (quantis calculados no cliente).

- **Alertmanager**

Recebe alertas do Prometheus e cuida de agrupamento, deduplicação, roteamento (e-mail, chat, plantão), silenciamento e inibição.

- **Grafana**

Plataforma de visualização com painéis e dashboards sobre várias fontes de dados (Prometheus, Loki, Elasticsearch, bancos SQL e outras), variáveis de dashboard e alertas. Porta padrão 3000. Loki é o agregador de logs, e Tempo, o de traces, do mesmo ecossistema.

- **Zabbix**

Servidor, agentes e proxies. Agente passivo: o servidor consulta o agente (porta 10050). Agente ativo: o agente envia os dados ao servidor (porta 10051). Itens coletam valores, triggers definem condições de problema, ações notificam e templates padronizam. Suporta SNMP, IPMI e JMX (este por meio do Zabbix Java Gateway), além de descoberta automática (LLD).

PROMETHEUS PUXA, ZABBIX TAMBÉM PODE RECEBER

Prometheus usa o modelo pull. O Zabbix aceita consulta pelo servidor (passivo) e envio pelo agente (ativo). A diferença entre as portas 10050 (agente) e 10051 (servidor) é cobrada com frequência.

13. Elastic Stack (ELK), APM e OpenTelemetry

A pilha Elastic (ELK) trata a ingestão, o armazenamento, a busca e a visualização de logs e outros dados. O APM e o rastreamento distribuído acompanham o desempenho das aplicações.

- **Componentes do ELK**

Elasticsearch (motor de busca e análise distribuído, com API REST na porta 9200 e comunicação entre nós na 9300), Logstash (pipeline de ingestão com entrada, filtro e saída; porta 5044 para Beats), Kibana (interface web, porta 5601) e Beats (agentes leves de coleta, como Filebeat e Metricbeat).

- **Índices, shards e réplicas**

Os dados ficam em índices divididos em shards (fragmentos) primários, distribuídos pelos nós. O número de shards primários é definido na criação do índice; o de réplicas pode mudar depois e dá tolerância a falhas e mais capacidade de leitura.

- **Ciclo de vida do índice (ILM)**

Move os índices por fases (hot, warm, cold, frozen e exclusão) conforme idade e tamanho, equilibrando custo e desempenho.

- **OpenSearch**

É um fork do Elasticsearch e do Kibana mantido pela comunidade e pela AWS, resultado da mudança de licença do Elastic.

- **APM e rastreamento distribuído**

APM (Application Performance Monitoring) mede tempo de resposta, vazão (throughput), taxa de erros e percentis (p95 e p99, mais úteis que a média). Um trace é composto por spans (trechos com início e duração) em relação pai e filho; o contexto é propagado entre serviços por cabeçalhos, como o traceparent do W3C Trace Context.

- **Apdex**

Índice de satisfação de 0 a 1: $(\text{satisfeitas} + \text{toleráveis}/2)$ dividido pelo total de requisições, com base em um tempo limite T (satisfeita até T, tolerável até 4T).

- **OpenTelemetry**

Projeto da CNCF que padroniza a geração e o envio de traces, métricas e logs: APIs, SDKs, instrumentação automática e manual, Collector e o protocolo OTLP, independente de fornecedor.

MÉDIA ESCONDE, PERCENTIL REVELA

A média de latência pode parecer boa enquanto poucos usuários sofrem esperas longas. Percentis altos (p95, p99) mostram essa cauda. Em APM, use percentis.

14. Aprofundamento: Arquitetura Orientada a Eventos, gRPC e Padrões de Resiliência

Complementando a mensageria já vista (ActiveMQ, Kafka), esta seção aprofunda o estilo arquitetural que se apoia nesses middlewares (arquitetura orientada a eventos), um protocolo de comunicação síncrona de alto desempenho entre serviços (gRPC) e os padrões de resiliência aplicados na integração entre sistemas.

- **Arquitetura Orientada a Eventos (Event-Driven Architecture, EDA)**

Estilo em que os serviços se comunicam publicando e reagindo a eventos (fatos que já aconteceram, como 'PedidoCriado' ou 'DocumentoFiscalEmitido'), em vez de chamarem diretamente uns aos outros. Produtores e consumidores ficam desacoplados no tempo e no espaço, e o broker de mensageria (ActiveMQ, Kafka) intermedia essa comunicação. Contrasta com a arquitetura orientada a requisição-resposta, em que um serviço chama outro diretamente e aguarda a resposta.

- **Event Notification, Event-Carried State Transfer e Event Sourcing**

Event Notification: o evento carrega só um identificador mínimo, e quem recebe precisa consultar o serviço de origem para obter os detalhes. Event-Carried State Transfer: o evento já carrega os dados relevantes, evitando uma nova chamada de consulta, ao custo de eventos maiores e de possível duplicação de dados. Event Sourcing: o estado de uma entidade não é armazenado diretamente; ele é reconstruído a partir da sequência completa de eventos já ocorridos, dando um histórico auditável nativo, característica valiosa para sistemas fiscais.

- **gRPC (Google Remote Procedure Call)**

Framework de comunicação síncrona entre serviços baseado em HTTP/2 e Protocol Buffers (serialização binária compacta, mais eficiente que JSON em texto). Suporta chamadas unárias (uma requisição, uma resposta) e streaming bidirecional. Mais indicado para comunicação interna de alto desempenho entre microserviços do que para APIs públicas voltadas a navegadores, onde REST/JSON ainda predomina pela simplicidade e interoperabilidade universal.

- **Padrões de Resiliência na Integração (Enterprise Integration Patterns)**

Circuit Breaker: interrompe temporariamente chamadas a um serviço que está falhando repetidamente, evitando falha em cascata. Retry com Backoff Exponencial: tenta novamente uma chamada falha, aumentando progressivamente o intervalo entre tentativas para não sobrecarregar um serviço já degradado. Bulkhead: isola os recursos (pools de conexão, threads) usados para chamar diferentes serviços dependentes, para que a lentidão de um deles não esgote os recursos disponíveis para os demais, à semelhança dos anteparos estanques de um navio.

ALERTA DE PROVA - EVENT SOURCING DÁ AUDITORIA NATIVA

Diferente do modelo tradicional (que só guarda o estado atual), o Event Sourcing guarda TODOS os eventos que já ocorreram; isso permite reconstruir o estado em qualquer ponto do passado, vantagem direta para sistemas que precisam de trilha de auditoria fiscal completa!

15. Referências e Conferência

Conferência realizada em 21/09/2026, com aprofundamento em 22/09/2026 (seção 14). Apostila criada a partir do programa SEFAZ-CE 2026 (FCC), área B02.

- **Referências**

Documentação oficial de WildFly e JBoss EAP; Eclipse Jakarta EE; Apache ActiveMQ (Classic e Artemis); Apache Kafka; Debezium; Kubernetes; Ansible; Helm; Podman e Open Container Initiative; Prometheus; Grafana; Zabbix; Elastic e OpenSearch; OpenTelemetry e W3C Trace Context; Hohpe e Woolf, Enterprise Integration Patterns; Google SRE Book; Edital SEFAZ-CE 2026 (FCC), Anexo VI.

- **A conferir**

Situação do ZooKeeper e do modo KRaft na versão do Kafka em uso; portas e valores padrão na versão instalada (podem mudar entre versões); nomes de subsistemas e arquivos de configuração no WildFly ou JBoss EAP instalado; itens de Debezium por banco de dados (suporte e requisitos variam por versão).

- **Lacunas conhecidas**

Padrões de microsserviços (BFF, CQRS, Saga, Circuit Breaker) serão tratados na apostila de Desenvolvimento e Arquiteturas. Integrações específicas de nuvem (AWS, Azure e GCP) ficam na apostila de Nuvem.

REGRA DE CONTEÚDO

Este material não afirma nada sem fonte. Se houver divergência com a documentação oficial da versão em uso, a documentação prevalece.

GLOSSÁRIO DE SIGLAS E ABREVIATURAS

Consulte abaixo o significado e a expansão explicativa de todas as siglas contábeis, tributárias e de TI presentes nas apostilas de preparação:

ACID	Atomicidade, Consistência, Isolamento e Durabilidade. Propriedades fundamentais para transações em SGBDs.
BI	Business Intelligence. Inteligência de Negócios para mineração e análise estratégica de dados.
CAP	Consistência, Disponibilidade e Tolerância a Partições. Teorema limitador para sistemas distribuídos.
CEBRASPE	Centro de Seleção e de Promoção de Eventos. Banca examinadora oficial de concursos públicos.
CFC	Conselho Federal de Contabilidade. Órgão responsável pela edição das NBCs no território nacional.
CIAP	Controle de Crédito de ICMS do Ativo Permanente. Controle fiscal de apropriação de créditos de bens imobilizados.
CID	Confidencialidade, Integridade e Disponibilidade. Pilares de sustentação da Segurança da Informação.
COBIT	Control Objectives for Information and Related Technologies. Framework global de Governança de TI.
CPC	Comitê de Pronunciamentos Contábeis. Emissor de normas convergentes com padrões internacionais.
FGV	Fundação Getulio Vargas. Banca examinadora de certames e instituição de pesquisa econômica.
ICMS	Imposto sobre Operações Relativas à Circulação de Mercadorias e Prestações de Serviços de Transporte.
IPVA	Imposto sobre a Propriedade de Veículos Automotores. Tributo de competência estadual.
ITD	Imposto sobre Transmissão Causa Mortis e Doação. Tributo sobre heranças e doações na Bahia.
ITIL	Information Technology Infrastructure Library. Boas práticas de Gerenciamento de Serviços de TI.
KDD	Knowledge Discovery in Databases. Processo sistemático de descoberta de conhecimento em bases de dados.
LDO	Lei de Diretrizes Orçamentárias. Define as metas e prioridades para o orçamento do exercício seguinte.
LOA	Lei Orçamentária Anual. Estima as receitas e fixa as despesas públicas para o exercício financeiro.
NBC TA	Normas Brasileiras de Contabilidade Técnicas de Auditoria Independente. Alinhadas ao padrão internacional.
NBC TI	Normas Brasileiras de Contabilidade Técnicas de Auditoria Interna. Foco nos processos organizacionais.
NoSQL	Not Only SQL. Família de SGBDs não relacionais estruturados para alto desempenho e escalabilidade.
OLAP	Online Analytical Processing. Ferramentas analíticas baseadas em cubos de dados multidimensionais.
PAF-BA	Processo Administrativo Fiscal do Estado da Bahia. Lei nº 3.956/1981 que regula o contencioso.
PPA	Plano Plurianual. Planejamento estratégico governamental de médio prazo (4 anos).
RICMS-BA	Regulamento do ICMS do Estado da Bahia. Decreto regulamentador do principal imposto estadual.
RUP	Rational Unified Process. Processo de desenvolvimento de software disciplinado e preditivo.
SEFAZ-BA	Secretaria da Fazenda do Estado da Bahia. Órgão de arrecadação e controle fiscal baiano.

SGBD	Sistema de Gerenciamento de Banco de Dados. Software de controle e armazenamento seguro de dados.
SQL	Structured Query Language. Linguagem declarativa padrão para consulta em SGBDs relacionais.
TI	Tecnologia da Informação. Recursos de hardware, software e infraestrutura de rede corporativa.
XP	Extreme Programming. Metodologia ágil de engenharia de software focada em excelência técnica.